

Relazione Tecnica

QSVM 6 qubit sul dataset Bank Account Fraud (BAF)

Giuseppe Pasquale Caligiure MAT:280867,
Ettore Aloe MAT:281116,
Nicola Mercuri MAT:276966

8 giugno 2026

Indice

1	Struttura del progetto	3
2	Blocco 1 – Caricamento del dataset	3
2.1	Il dataset BAF (Bank Account Fraud) – NeurIPS 2022	3
3	Blocco 2 – Preprocessing	5
4	Blocco 3 – Mutual Information e scelta della top feature	5
5	Blocco 4 – PCA a 5 componenti + feature supervisionata	6
6	Blocco 5 – Bilanciamento delle classi	6
7	Blocco 6 – Scaling angolare per la ZZFeatureMap	7
8	Blocco 7 – Train/Test split	7
9	Blocco 8 – Quantum kernel con Qiskit	7
9.1	ZZFeatureMap a 6 qubit	7
9.2	Backend quantistico	8
9.3	Compute–uncompute per il kernel	8
9.4	Matrici K_train e K_test	8
10	Blocco 9 – QSVM e baseline classiche	9
11	Blocco 10 – Grafici e visualizzazioni	9
11.1	Confronto delle metriche QSVM vs SVM classiche	9
11.2	Confusion matrices	10
11.3	Kernel concentration analysis	10
12	Blocco 11 – Salvataggio risultati e file di output	11
13	Considerazioni metodologiche e interpretative	11
13.1	Motivazioni della scelta PCA(5) + feature MI	11
13.2	Confronto dei risultati: 6 vs 7 vs 8 qubit	12
13.3	Confronto dei risultati: ZZFeatureMap vs PauliFeatureMap (XYZZ) a 6 qubit	13
13.4	Confronto dei risultati: ZZFeatureMap (fidelity globale) vs Projected Quantum Kernel (PQK)	14

1 Struttura del progetto

1. Caricamento e pulizia del dataset BAF.
2. Preprocessing classico (encoding, imputazione, standardizzazione).
3. Calcolo della **Mutual Information** (MI) per valutare l'importanza delle feature rispetto al target.
4. Riduzione dimensionale tramite **PCA a 5 componenti**.
5. Selezione della **migliore feature supervisionata** (top-MI) e costruzione di uno spazio finale a **6 dimensioni**.
6. Bilanciamento delle classi e riscalamento angolare in $[-\pi, \pi]$ per l'input della ZZFeatureMap a 6 qubit.
7. Costruzione del **kernel quantistico** tramite circuito compute-uncompute con Qiskit Aer.
8. Addestramento della **QSVM** (SVC con `kernel="precomputed"`) e confronto con SVM lineare e SVM RBF classiche.
9. Analisi quantitativa e grafica dei risultati, con salvataggio di metriche e matrici di confusione.

L'intero flusso è implementato nel file Python:

- `qsvm_baf_6qubit_pca5_mi1.py`

2 Blocco 1 – Caricamento del dataset

2.1 Il dataset BAF (Bank Account Fraud) – NeurIPS 2022

Il dataset utilizzato in questo progetto è il **Bank Account Fraud Dataset Suite (BAF)**, pubblicato come contributo alla conferenza NeurIPS 2022 e disponibile su [Kaggle](#).

Si tratta di un dataset **sintetico ma realistico**, generato a partire da dati reali di un istituto bancario tramite tecniche di sintesi controllata, con l'obiettivo di fornire un banco di prova standard per la valutazione di algoritmi di rilevazione delle frodi. La suite BAF è composta da un file base (**Base.csv**) e cinque varianti (**Variant I-V**), ciascuna arricchita con specifici tipi di data bias per testare la robustezza e l'equità (fairness) dei modelli. In questo progetto viene utilizzato esclusivamente il file **Base.csv**.

Struttura e dimensioni

Caratteristica	Valore
Numero di istanze	1.000.000
Numero di feature	31 (+ 1 colonna target)
Colonna target	<code>fraud_bool</code> (0 = legittimo, 1 = frode)

Caratteristica	Valore
Tipo di dati	Misto: numerici continui, interi, categorici
Valori mancanti	Codificati come -1 in alcune colonne

Le feature coprono diverse dimensioni comportamentali e demografiche del richiedente: reddito (`income`), età (`customer_age`), storico degli indirizzi (`prev_address_months_count`, `current_address_months_count`), velocità di richiesta (`velocity_6h`, `velocity_24h`, `velocity_4w`), punteggio di rischio interno (`credit_risk_score`), informazioni sul dispositivo (`device_os`, `device_distinct_emails_8w`), canale di accesso (`source`), e attributi protetti (gruppo d'età, stato occupazionale, percentuale di reddito) per consentire valutazioni di fairness.

Lo sbilanciamento delle classi: il problema centrale La caratteristica più critica del dataset è il suo **fortissimo sbilanciamento tra le classi**:

- **Classe 0 (legittimo):** ~988.900 istanze — circa **98.90%** del totale
- **Classe 1 (frode):** ~11.100 istanze — circa **1.10%** del totale

Questo rapporto di circa **1:90** tra frodi e richieste legittime rispecchia la distribuzione reale del fenomeno, ma pone una serie di sfide metodologiche significative per qualsiasi algoritmo di classificazione:

1. **Bias verso la classe maggioritaria.** Un classificatore banale che predice sempre "legittimo" ottiene un'accuracy del ~98.90% senza apprendere nulla di utile. Metriche come l'*accuracy* diventano quindi del tutto fuorvianti come indicatori di performance reale.
2. **Scarsità di esempi positivi per l'apprendimento.** Con soli ~11.100 esempi di frode su 1 milione, il modello dispone di pochissimi pattern fraudolenti da cui generalizzare. Questo è particolarmente problematico per algoritmi come le SVM, che dipendono dalla struttura geometrica dello spazio delle feature per definire il confine decisionale.
3. **Metriche di valutazione inadeguate.** Per quantificare correttamente le performance in presenza di sbilanciamento è necessario ricorrere a metriche specifiche come **Precision**, **Recall**, **F1-score** e **ROC-AUC**, che tengono conto separatamente degli errori su ciascuna classe. In particolare, il **Recall sulla classe frode** è la metrica più critica in ambito applicativo: un falso negativo (frode non rilevata) ha conseguenze economiche molto più gravi di un falso positivo (richiesta legittima bloccata).
4. **Necessità di bilanciamento artificiale.** Per consentire un addestramento efficace, è indispensabile intervenire sulla distribuzione delle classi. Nel nostro script questo viene realizzato nel **Blocco 5**, dove viene estratto un sotto-campione bilanciato di **150 esempi per classe** (300 totali), applicando oversampling con rimpiazzo sulla classe minoritaria se necessario. Questo approccio, pur riducendo la quantità complessiva di dati utilizzati, garantisce

che il modello riceva un segnale di apprendimento equilibrato da entrambe le classi.

5. **Rischio di overfitting sul campione bilanciato.** Lavorando su soli 300 campioni estratti da 1 milione, il modello potrebbe non catturare tutta l'eterogeneità del fenomeno fraudolento. Questo è un limite intrinseco dell'approccio adottato, giustificato dall'elevato costo computazionale del kernel quantistico, e deve essere tenuto presente nell'interpretazione dei risultati.

Questo blocco è quindi puramente diagnostico: verifica che i dati siano stati caricati correttamente da `Base.csv` e stampa a console le distribuzioni delle classi, confermando che lo sbilanciamento osservato (~1.10% di frodi) sia in linea con le aspettative prima di procedere con il preprocessing e il bilanciamento artificiale.

3 Blocco 2 – Preprocessing

Il preprocessing prepara i dati per le fasi successive di feature selection e proiezione. Le operazioni includono:

- Rimozione di colonne non utilizzabili
 - Encoding delle variabili categoriche: tutte le colonne di tipo `object` o `category` (eccetto `fraud_bool`) vengono trasformate con `LabelEncoder`, ottenendo feature numeriche intere.
 - Conversione numerica forzata: garantisce che **tutte** le feature in `X` siano numeriche e compatibili con PCA e MI.
 - Imputazione dei valori mancanti: si calcola la mediana **escludendo i -1**, si sostituiscono i `-1` con la mediana. Questa scelta evita che il valore `-1` sposti la stima del valore tipico verso il basso e rappresenta una strategia robusta per l'imputazione di dati mancanti.
 - Separazione feature/target e standardizzazione
-

4 Blocco 3 – Mutual Information e scelta della top feature

In questo blocco si calcola la **Mutual Information** fra ogni feature e il target `fraud_bool` (usando `mutual_info_classif` di `scikit-learn`).

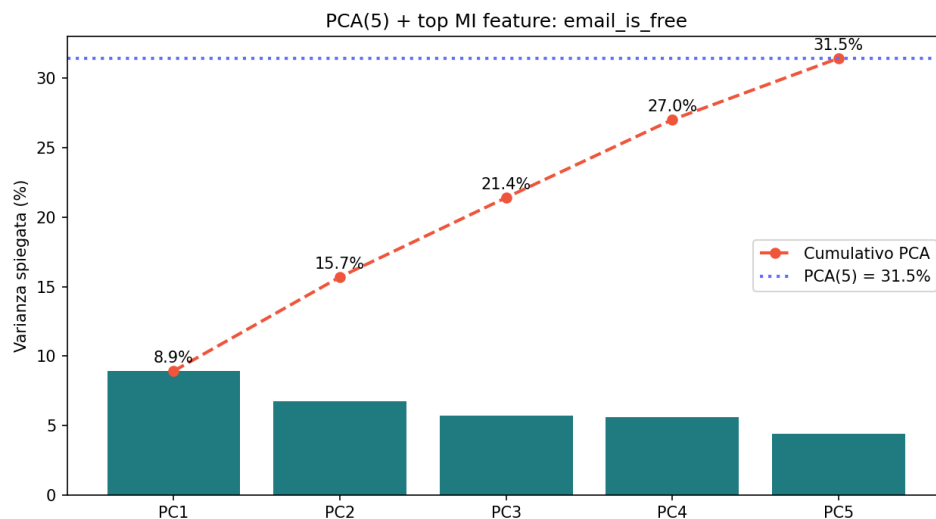
Questa feature a massima informazione mutua sarà poi preservata esplicitamente come **sesto input** della rappresentazione a 6 dimensioni usata dalla QSVM.

5 Blocco 4 – PCA a 5 componenti + feature supervisionata

L'obiettivo di questo blocco è costruire uno spazio di input a **6 dimensioni** che combini:

- **5 componenti principali PCA** calcolate su tutte le feature standardizzate.
- **1 feature supervisionata** (la `top_mi_feature`) che ha la massima dipendenza informativa con il target.

PCA (Principal Component Analysis) è una trasformazione lineare che permette di ridurre la dimensionalità del dataset, preservando la varianza spiegata. La sua funzione è quella di proiettare i dati su un nuovo sistema di assi cartesiani ortogonali, detti "componenti principali", in modo tale che la varianza dei dati sia massimizzata lungo queste direzioni.



Per la costruzione dello spazio a 6 dimensioni si estraggono i valori della `top_mi_feature` dal DataFrame standardizzato `X_scaled_df` e li si concatena alle 5 componenti PCA.

Questo compromesso aumenta la varianza spiegata rispetto ad una PCA a 4 componenti e, allo stesso tempo, mantiene una dimensione supervisionata fortemente predittiva.

6 Blocco 5 – Bilanciamento delle classi

Per ridurre lo sbilanciamento e contenere il costo computazionale del kernel quantistico, lo script lavora su un **sotto-campione bilanciato** del dataset.

1. Si crea un DataFrame `df_6` che contiene le 6 feature finali e il target `fraud_bool`.
2. Si separano i casi legittimi (`fraud_bool = 0`) e fraudolenti (`fraud_bool = 1`).
3. Si campionano **150 esempi per ciascuna classe** (300 in totale), usando `resample` con `replace=False` per la classe maggioritaria e `replace=True` se la classe minoritaria dispone di meno di 150 osservazioni.
4. Si mescolano i 300 esempi ottenuti e si estraggono `X_bal` (feature) e `y_bal` (target).

Questo setup mantiene il problema relativamente piccolo (permette di calcolare il kernel quantistico in tempi ragionevoli) e garantisce classi bilanciate per l'addestramento.

7 Blocco 6 – Scaling angolare per la ZZFeatureMap

```
angle_scaler = MinMaxScaler(feature_range=(-np.pi, np.pi))  
X_angle = angle_scaler.fit_transform(X_bal)
```

Dopo questa trasformazione, ogni dimensione del vettore è un angolo in radianti, pronto per essere fornito alla ZZFeatureMap a 6 qubit.

8 Blocco 7 – Train/Test split

Si esegue uno split **75/25 stratificato** sul dataset bilanciato:

- `X_train` contiene 225 campioni.
 - `X_test` contiene 75 campioni.
 - Le classi rimangono bilanciate in entrambi i sottoinsiemi.
-

9 Blocco 8 – Quantum kernel con Qiskit

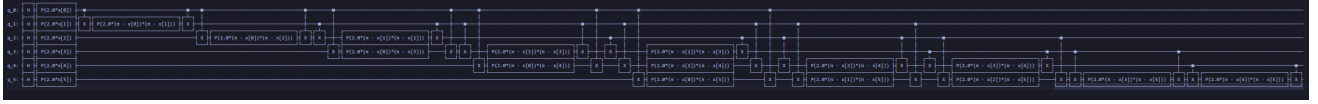
Questo è il cuore quantistico della pipeline. Lo script costruisce una ZZFeatureMap a 6 qubit e utilizza il **formalismo compute-uncompute** per stimare una matrice kernel di tipo fidelity.

9.1 ZZFeatureMap a 6 qubit

La feature map è definita come:

```
feature_map = ZZFeatureMap(feature_dimension=N_QUBITS, reps=1, entanglement="full")
```

- `feature_dimension = 6` perché lo spazio di input ha 6 dimensioni.
- `reps = 1` per mantenere la profondità del circuito contenuta.
- `entanglement = "full"` per permettere interazioni ZZ fra tutti i qubit, aumentando l'espressività del kernel.



9.2 Backend quantistico

Lo script utilizza `AerSimulator` con metodo `statevector` e tenta prima di usare la GPU (`device="GPU"`). Se questa non è disponibile, effettua automaticamente il fallback su CPU.

Il numero di shots per stimare ciascuna entry del kernel è `N_SHOTS = 8192`, un compromesso fra precisione statistica e tempo di esecuzione.

9.3 Compute–uncompute per il kernel

Per due vettori di input x_i, x_j (già angularizzati) si costruisce il circuito.

Lo script definisce due funzioni:

- `compute_kernel_entry(xi, xj, fmap, backend, shots)` – calcola una singola entry.
- `compute_kernel_matrix(X_a, X_b, fmap, backend, shots, symmetric)` – calcola un'intera matrice kernel fra i vettori di `X_a` e `X_b`, sfruttando la simmetria quando `X_a` e `X_b` coincidono.

9.4 Matrici `K_train` e `K_test`

Si calcola innanzitutto la matrice kernel del train set:

```
K_train = compute_kernel_matrix(X_train, X_train, feature_map, backend,
symmetric=True)
```

Questa matrice è **simmetrica** per costruzione e viene verificata numericamente.

Successivamente si calcola la matrice kernel fra test e train:

```
K_test = compute_kernel_matrix(X_test, X_train, feature_map, backend, sym-
metric=False)
```

10 Blocco 9 – QSVM e baseline classiche

Una volta ottenute K_{train} e K_{test} , lo script addestra e valuta:

1. **QSVM (6 qubit: PCA5 + MI1)** – un SVC con `kernel="precomputed"` usando direttamente le matrici kernel quantistiche.
2. **SVM lineare** – un SVC classico con `kernel="linear"` addestrato sui vettori X_{train} (6 dimensioni).
3. **SVM RBF** – un SVC classico con `kernel="rbf"` addestrato sugli stessi vettori.

Per ciascun modello vengono calcolate le principali metriche di classificazione sul test set:

- Accuracy.
- Precision.
- Recall.
- F1-score.
- ROC-AUC (tramite probabilità predette).

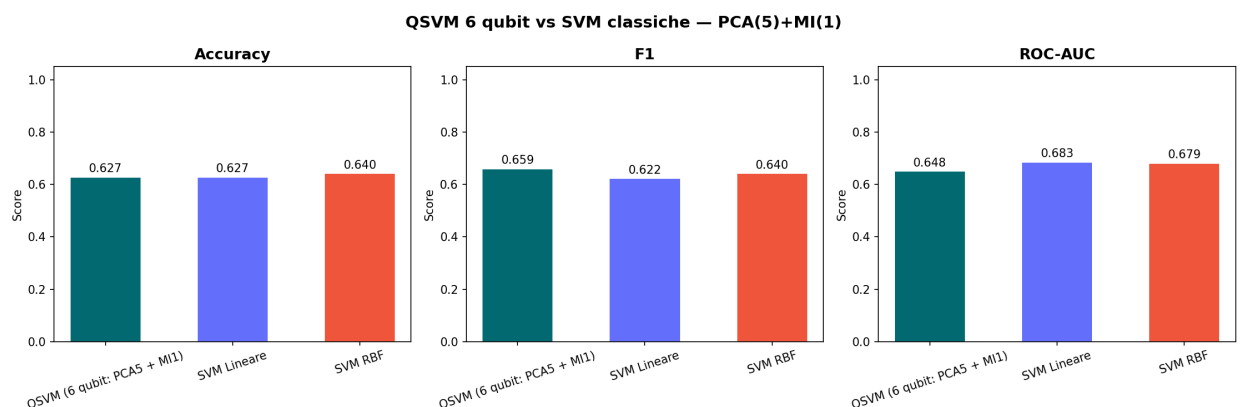
Questo consente un confronto diretto tra le prestazioni della QSVM e delle SVM classiche sullo **stesso spazio di input a 6 dimensioni**.

11 Blocco 10 – Grafici e visualizzazioni

Per una comprensione più intuitiva dei risultati, lo script genera diversi grafici.

11.1 Confronto delle metriche QSVM vs SVM classiche

Viene generato un grafico a barre per **Accuracy**, **F1** e **ROC-AUC** (`benchmark_metrics_6q.png`).



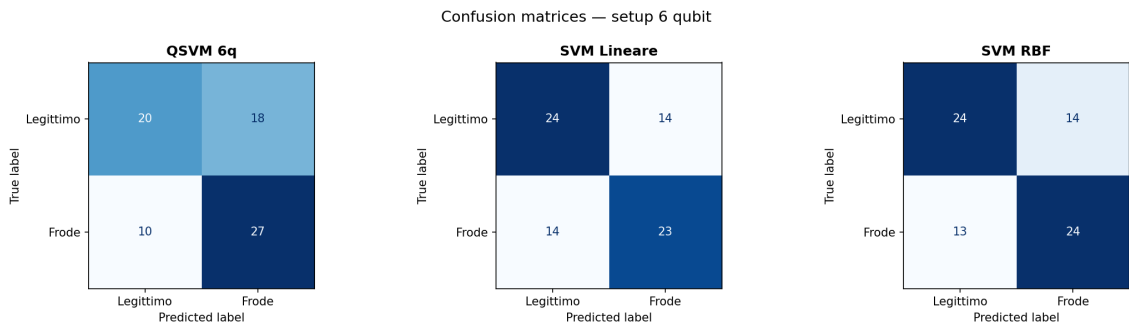
La QSVM viene confrontata con:

- una SVM lineare (margine lineare nello spazio a 6 dimensioni);
- una SVM RBF (kernel non lineare classico).

In molti scenari il modello SVM RBF ottiene prestazioni competitive o superiori rispetto alla QSVM, a fronte di un costo computazionale significativamente inferiore. Questo risultato mette in luce che il **vantaggio quantistico non è garantito a priori** e deve essere verificato sperimentalmente caso per caso.

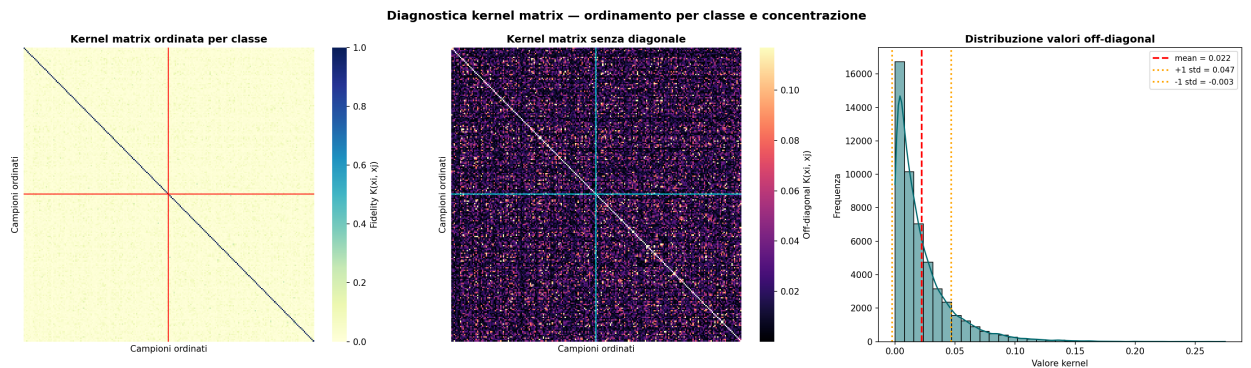
11.2 Confusion matrices

Vengono create tre confusion matrices affiancate (`confusion_matrices_6q.png`), una per ciascun modello.



Questa visualizzazione permette di capire **come** ciascun modello commette errori (es. se tende a produrre molti falsi positivi o falsi negativi).

11.3 Kernel concentration analysis



- Pannello 1 (con diagonale, scala 0–1): ci dice che la diagonale è vicina a 1 e che i valori tra punti diversi sono molto più bassi. Il pannello 1 è la heatmap della kernel matrix completa. La diagonale principale è ben visibile: rappresenta $K(x_i, x_i)$, quindi valori di auto-similarità molto alti, prossimi a 1. Ogni punto della mappa è il valore di $K(x_i, x_j)$ fra due campioni del train set.
- Pannello 2 (senza diagonale, scala ristretta): mostra la struttura fine dentro i valori piccoli. Mascherare la diagonale evita che i valori vicini a 1 dominino la colormap; restringere la scala ai percentile 1–99 degli off-diagonal aumenta il contrasto nelle zone che servono per diagnosticare la concentration. Il fatto che appaiano elementi colorati e non un unico colore piatto indica che c'è ancora variabilità locale, segno che il kernel non è collassato su una costante.
- Pannello 3 (distribuzione dei valori off-diagonal): ogni barra è il numero di coppie (x_i, x_j) con $i \neq j$ che hanno un certo valore di $K(x_i, x_j)$. Le barre sono molto concentrate vicino a 0 con una coda lunga verso destra (fino a circa 0.25), quindi la maggior parte delle coppie è poco simile e solo poche hanno similarità più alta. In termini di concentration, una media bassa, una deviazione standard non trascurabile e una coda evidente verso destra indicano che i valori off-diagonal non si schiacciano tutti su un unico valore; il kernel distingue ancora tra coppie quasi ortogonali (valore vicino a 0) e coppie più affini (valori maggiori).

12 Blocco 11 – Salvataggio risultati e file di output

Al termine dell'esecuzione, lo script salva i file che documentano la parte algoritmica e sperimentale del progetto.

13 Considerazioni metodologiche e interpretative

13.1 Motivazioni della scelta PCA(5) + feature MI

Inizialmente abbiamo testato 4 componenti PCA, ma abbiamo visto che non era sufficiente per catturare abbastanza informazione, infatti l'accuracy era molto bassa. Allora abbiamo deciso di provare la combinazione **PCA(5)** (più varianza spiegata totale) e **feature supervisionata top-MI** (dimensione fortemente correlata con il target), che mira a ridurre la perdita di informazione discriminativa prima della codifica quantistica. I primi 5 qubit rappresentano una proiezione lineare non supervisionata, mentre il sesto qubit conserva una variabile esplicitamente significativa.

13.2 Confronto dei risultati: 6 vs 7 vs 8 qubit

Per esplorare l'impatto dell'aumento della dimensionalità (e quindi dei qubit), sono stati condotti tre esperimenti utilizzando 6, 7 e 8 qubit. In ogni esperimento l'architettura dei dati è stata mantenuta coerente: una singola feature supervisionata a massima Mutual Information, e le restanti dimensioni costituite dalle componenti principali (PCA).

I risultati ottenuti dalle rispettive QSVM sui dataset di test (addestrati con parametri e bilanciamento analoghi) sono riassunti di seguito:

Metrica	QSVM 6 Qubit (PCA5 + MI1)	QSVM 7 Qubit (PCA6 + MI1)	QSVM 8 Qubit (PCA7 + MI1)
Accuracy	0.627	0.600	0.573
Precision	0.600	0.590	0.558
Recall	0.730	0.622	0.649
F1-Score	0.659	0.605	0.600
ROC-AUC	0.648	0.364	0.612

Contrariamente all'intuizione classica per cui aggiungere feature aumenta la capacità del modello, nei nostri test **l'aumento del numero di qubit ha portato a un progressivo peggioramento delle performance**. Il modello a 6 qubit rimane nettamente il migliore.

Perché aumentare i qubit ha peggiorato i risultati? Questo fenomeno controintuitivo è ben noto nel Quantum Machine Learning e deriva dai seguenti motivi interconnessi:

1. **Kernel Concentration (Exponential Concentration):** L'utilizzo della `ZZFeatureMap` con entanglement "full" mappa i dati in uno spazio di Hilbert la cui dimensione cresce esponenzialmente. Aumentando il numero di qubit da 6 a 8, la distanza tra i vettori di stato nello spazio quantistico si massimizza. La conseguenza è che i prodotti interni (fidelity) tra stati differenti tendono tutti a zero, facendo collassare la matrice kernel verso la matrice identità. La QSVM perde così la sua capacità di generalizzare in quanto tutti i punti appaiono ortogonali ed equidistanti fra loro.
2. **"Rumore" dalle componenti PCA aggiuntive:** Le componenti PCA aggiunte nei modelli a 7 e 8 qubit (la 6^a e 7^a componente) catturano frazioni sempre minori di varianza utile rispetto alle prime. In una codifica non lineare altamente sensibile come quella quantistica (dove i valori diventano angoli di rotazione dei qubit), queste feature a bassa varianza introducono primariamente "rumore quantistico" che disturba la codifica delle feature principali, peggiorando la separazione delle classi.
3. **Maledizione della Dimensionalità Quantistica (*Curse of Dimensionality*):** Il volume dello spazio degli stati quantistici cresce così velocemente che, mantenendo invariata la quantità di dati di addestramento (300 campioni in totale per il training set), la densità dei punti nel modello a 8 qubit si dirada

drasticamente. Questo spinge il modello a "memorizzare" il dataset di training (overfitting) e comporta un calo delle performance di generalizzazione sul set di test (come evidenziato dalla caduta dell'Accuracy).

Per queste ragioni metodologiche, il modello a 6 qubit rappresenta per il nostro problema il miglior compromesso ("sweet spot") tra la ricchezza dell'informazione iniettata e la mitigazione degli effetti negativi legati all'alta dimensionalità.

13.3 Confronto dei risultati: ZZFeatureMap vs PauliFeatureMap (XYZZ) a 6 qubit

Per valutare l'impatto del tipo di codifica quantistica, è stato effettuato un test mantenendo fisso il numero di qubit a 6 (PCA5 + MI1) e sostituendo la ZZFeatureMap standard con una più complessa PauliFeatureMap basata sugli operatori di Pauli X, Y e ZZ.

I risultati ottenuti dalle due diverse feature map sul medesimo set di test sono i seguenti:

Metrica	ZZFeatureMap (6 Qubit)	PauliFeatureMap (X, Y, ZZ - 6 Qubit)
Accuracy	0.627	0.560
Precision	0.600	0.563
Recall	0.730	0.486
F1-Score	0.659	0.522
ROC-AUC	0.648	0.520

Il passaggio alla PauliFeatureMap ha causato un **netto deterioramento delle prestazioni**, riducendo la capacità predittiva del modello quasi a livello di una scelta casuale (ROC-AUC ~0.52 contro 0.648).

Perché la PauliFeatureMap ha peggiorato i risultati? Questo crollo di performance è strettamente legato alle dinamiche interne dei circuiti quantistici variazionali:

1. **Eccessiva Espressività (Over-parameterization):** Mentre la ZZFeatureMap impiega principalmente layer di Hadamard seguiti da entanglement ZZ, l'inclusione sistematica di rotazioni X e Y nella PauliFeatureMap rende il circuito estremamente più espressivo. Nel Quantum Machine Learning, una feature map troppo espressiva tende a comportarsi come un'operazione pseudo-casuale, spalmando uniformemente i dati in tutto l'immenso spazio di Hilbert. Ne consegue che ogni coppia di dati viene mappata su stati quasi perfettamente ortogonali: la matrice kernel collassa quindi su una matrice identità (esasperando la *Kernel Concentration*) annullando la capacità dell'algoritmo di misurare la vera somiglianza ("fidelity") tra i campioni.
2. **Disallineamento con la struttura dei dati (Feature Mapping Mismatch):** Per i dati tabulari standardizzati del dataset BAF, l'interferenza

generata dalla **ZZFeatureMap** riesce a cogliere e codificare in modo più robusto le differenze significative. L'aggiunta di rotazioni lungo gli assi X e Y, senza una specifica motivazione legata al dominio del problema, altera pesantemente le fasi dei qubit agendo come vero e proprio "rumore quantistico", che scompone irrimediabilmente i pattern originari faticosamente appresi dal modello.

13.4 Confronto dei risultati: ZZFeatureMap (fidelity globale) vs Projected Quantum Kernel (PQK)

Per valutare un approccio alternativo alla costruzione del kernel quantistico, è stato implementato un **Projected Quantum Kernel (PQK)** che, pur utilizzando la stessa **ZZFeatureMap** a 6 qubit (PCA5 + MI) come encoder, sostituisce il formalismo compute–uncompute con una misura di **expectation values locali** per ogni qubit.

Come funziona il PQK Mentre il kernel di fidelity globale calcola la fidelity tramite un circuito doppio (compute–uncompute), il PQK:

1. **Proietta** ogni campione (x) in un vettore classico di dimensione n (numero di qubit), misurando l'expected value dopo l'encoding con la **ZZFeatureMap**.
2. **Costruisce** un kernel RBF classico su queste proiezioni
3. **Riduce** il costo computazionale da $O(n^2)$ a $O(n)$ circuiti quantistici.

Risultati a confronto I risultati ottenuti dai due approcci sullo stesso dataset di test (6 qubit, PCA5 + MI, 300 campioni bilanciati) sono i seguenti:

Metrica	QSVM ZZFeatureMap (fidelity globale)	QSVM PQK (expectation values + RBF)
Accuracy	0.627	0.493
Precision	0.600	0.493
Recall	0.730	1.000
F1-Score	0.659	0.661
ROC-AUC	0.648	0.431

Perché il PQK ha fallito in questo contesto? Il PQK mostra un comportamento anomalo: **Recall = 1.0** significa che predice **tutti i campioni come frode**, classificando erroneamente anche tutti i legittimi. Questo è confermato da **Accuracy = Precision = 0.493** (cioè circa il 49.3% del test set è della classe frode) e da un **ROC-AUC = 0.431**, inferiore alla soglia di casualità pura (0.5), che indica che il modello è peggio di un classificatore casuale.

Questo risultato, apparentemente paradossale (un approccio teoricamente più robusto che performa peggio), si spiega con una ragione strutturale: **il vantaggio teorico del PQK non si applica a questo regime.**

Il PQK è stato teorizzato per aggirare il problema del *kernel concentration* (collasso della matrice di fidelity verso l'identità) tipico dei kernel quantistici su molti qubit.

Tuttavia, con soli 6 qubit e **reps=1**, il kernel di fidelity globale **non** soffre ancora significativamente di questo problema (come dimostrato dal ROC-AUC = 0.648 della ZZFeatureMap). In questo regime a bassa dimensionalità, il kernel di fidelity riesce ancora a catturare differenze significative tra i campioni, mentre il PQK — pensato per circuiti più profondi e con più qubit — introduce una perdita di informazione non compensata da alcun beneficio.

In sintesi, il PQK è uno strumento valido per circuiti quantistici profondi e ad alto numero di qubit, dove il kernel di fidelity collassa. A 6 qubit con **reps=1**, il formalismo compute–uncompute della ZZFeatureMap preserva meglio la struttura informativa dei dati, risultando nettamente superiore.